

## CHAPTER 4

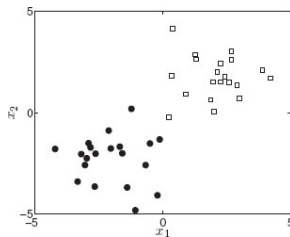
## Bayesian Inference

In the previous chapter we introduced the key concepts required to adopt a Bayesian approach to machine learning. Within the Bayesian framework, all unknown quantities are treated as random variables. Each parameter is described by a distribution rather than an individual value. Uncertainty in our parameter estimates is naturally channelled into any predictions we make. We saw two examples of *prior* and *likelihood* combinations that were *conjugate*, meaning that the *posterior* would be of the same form as the prior and could be computed analytically. Examples where we can justify the choice of a conjugate prior and likelihood combination are rare. In the remainder, we cannot compute the posterior and must resort to approximations. In this chapter, we will introduce three such approximation techniques.

## 4.1 NON-CONJUGATE MODELS

In the previous chapter we saw two models for which exact Bayesian inference was possible. In the first case, we were modelling the tossing of a coin and the combination of a beta prior and binomial likelihood meant that we could state that the posterior would also belong to the beta family. In the second example, a Gaussian prior coupled with a Gaussian likelihood resulted in a Gaussian posterior. The fact that we knew the form of the posterior meant that we didn't need to calculate the normalisation constant (the denominator in, for example, Equation 3.3). As long as we could find something proportional to the density of interest (i.e. proportional to a beta or a Gaussian), we could be certain that the normalisation would take care of itself. The beta-binomial and Gaussian-Gaussian combinations are not the only conjugate prior/likelihood pairs that we can use. Two other popular examples are the multinomial-Dirichlet and the gamma-Gaussian for discrete and continuous data, respectively.

For many models, it is not possible (or not justifiable from a modelling perspective) to pick a conjugate prior and likelihood, and we are forced to approximate. In this chapter, we will introduce three approximation techniques through a binary classification problem. Binary classification is a common problem within machine learning and one for which no conjugate prior and likelihood combination exists. The three techniques that we will look at are a point estimate, an approximate density, and sampling. All three are widely used within machine learning.



Rogers, Simon, and Mark Girolami. A First Course in Machine Learning, CRC Press LLC, 2016. ProQuest Ebook Central, <http://ebookcentral.proquest.com/lib/uzr/detail.action?docID=4718644>. Created from uzr on 2025-04-07 07:45:16.

$t_n$  is a binary variable indicating the class (0 or 1) of the  $n$ th object,  $\mathbf{x}_n$ . In the Gaussian Olympics example in the previous chapter, we treated  $t_n$  as a Gaussian random variable with mean  $\mathbf{w}^T \mathbf{x}_n$  and variance  $\sigma^2$ , but this is only appropriate for real-valued  $t_n$ . Instead, we can model  $t_n$  as a binary random variable – a single coin toss for each  $n$ . Rather than a mean and variance, this random variable is characterised by the probability that the class is 1 (the probability of belonging to class 0 is 1 minus the probability of belonging to class 1). To avoid confusion, we will denote this random variable  $T_n$  (to distinguish it from the actual instance,  $t_n$ , that we observe). Therefore, we can write each of the  $n$  likelihood terms as a probability:

$$p(\mathbf{t}|\mathbf{X}, \mathbf{w}) = \prod_{n=1}^N P(T_n = t_n | \mathbf{x}_n, \mathbf{w}) \quad (4.2)$$

This likelihood function will be high if the model assigns high probabilities for class 1 when we observe class 1 and high probabilities for class 0 when we observe class 0. It has a maximum value of 1 where all of the training points are predicted perfectly.

Our task is now to choose a function of  $\mathbf{x}_n$  and  $\mathbf{w}$ ,  $f(\mathbf{x}_n; \mathbf{w})$ , that produces a probability. A popular technique is to take a simple linear function (e.g.  $f(\mathbf{x}_n; \mathbf{w}) = \mathbf{w}^T \mathbf{x}_n$ ) and then pass the result through a second function that *squashes* its output to ensure it produces a valid probability. One such squashing function is the sigmoid function shown in Figure 4.2. As  $\mathbf{w}^T \mathbf{x}$  increases, the value converges to 1 and as it decreases, it converges to 0. The sigmoid function is defined as

$$P(T_n = 1 | \mathbf{x}_n, \mathbf{w}) = \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x}_n)} \quad (4.3)$$

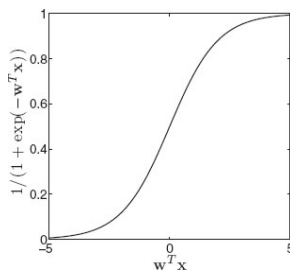


FIGURE 4.2 The sigmoid function that squashes a real value (e.g.  $\mathbf{w}^T \mathbf{x}$ ) to always be between 0 and 1.

This expression gives us the probability that  $T_n = 1$ . In our likelihood we require the probability of the actual observation, some of which will be zero. Because  $T_n$  can only take the value 0 or 1, we can easily compute  $P(T_n = 0 | \mathbf{x}_n, \mathbf{w})$  using Equation 2.2:

$$(4.4)$$

FIGURE 4.1 An example of a dataset with a binary response. Each object is defined by two attributes ( $x_1$  and  $x_2$ ) and a binary target,  $t = \{0, 1\}$ . Points with  $t = 0$  are plotted as circles and those with  $t = 1$  as squares.

## 4.2 BINARY RESPONSES

Figure 4.1 shows a dataset that looks a bit different from those we have seen so far. Each object is described by two attributes,  $x_1$  and  $x_2$ , and has a binary response,  $t = \{0, 1\}$ . The objects are plotted with a symbol that depends on their response: if  $t = 0$ , the point is plotted as a circle, and, if  $t = 1$ , as a square. We will use this data to build a model that will enable us to predict the response (0 or 1; circle or square) for a new object. This task is known as classification – we want to be able to classify objects into one of a set of classes (in this case there are two classes). Classification is one of the major problems within machine learning, and we will introduce several other classification algorithms in Chapter 5.

## 4.2.1 A model for binary responses

We will work with the following vector and matrix representations of our data:

$$\mathbf{x}_n = \begin{bmatrix} x_{n1} \\ x_{n2} \end{bmatrix}, \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}, \mathbf{X} = \begin{bmatrix} \mathbf{x}_1^T \\ \mathbf{x}_2^T \\ \vdots \\ \mathbf{x}_N^T \end{bmatrix}$$

Our model (with parameters  $\mathbf{w}$ ) will allow us to predict  $t_{\text{new}}$  for some new observation  $\mathbf{x}_{\text{new}}$ .

Just as in our Olympics example in Section 3.8, we will need to compute the posterior density over the parameters of the model. According to Bayes' rule, this is given by

$$p(\mathbf{w} | \mathbf{t}, \mathbf{X}) = \frac{p(\mathbf{t} | \mathbf{X}, \mathbf{w}) p(\mathbf{w})}{p(\mathbf{t})} \quad (4.1)$$

where the marginal likelihood  $p(\mathbf{t} | \mathbf{X})$  is given by

$$p(\mathbf{t} | \mathbf{X}) = \int p(\mathbf{t} | \mathbf{X}, \mathbf{w}) p(\mathbf{w}) d\mathbf{w}.$$

**Prior:** We shall use a Gaussian density for the prior,  $p(\mathbf{w})$ . In particular,  $p(\mathbf{w}) = \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$ . To be consistent, given that  $p(\mathbf{w})$  depends on  $\sigma^2$ , we will denote the prior as  $p(\mathbf{w} | \sigma^2)$ . In previous chapters, the choice of a Gaussian density was often motivated by analytical convenience. Given that we are not going to be able to rely on conjugacy in this chapter, we are not restricted in our choice of prior density. However, our interest in this chapter is in the methods required to overcome nonconjugacy and for that, a Gaussian will suffice. Readers are recommended to try the methods introduced in this chapter with different forms of prior density,  $p(\mathbf{w})$ .

**Likelihood:** To make headway with the likelihood,  $p(\mathbf{t} | \mathbf{X}, \mathbf{w})$ , we start by assuming that the elements of  $\mathbf{t}$  are conditionally independent (see Section 2.8.1), conditioned on  $\mathbf{w}$ :

$$p(\mathbf{t} | \mathbf{X}, \mathbf{w}) = \prod_{n=1}^N p(t_n | \mathbf{x}_n, \mathbf{w}).$$

Rogers, Simon, and Mark Girolami. A First Course in Machine Learning, CRC Press LLC, 2016. ProQuest Ebook Central, <http://ebookcentral.proquest.com/lib/uzr/detail.action?docID=4718644>. Created from uzr on 2025-04-07 07:45:16.

$$\begin{aligned} P(T_n = 1 | \mathbf{x}_n, \mathbf{w}) &= 1 - P(T_n = 0 | \mathbf{x}_n, \mathbf{w}) \\ &= \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x}_n)} \\ &= \frac{\exp(\mathbf{w}^T \mathbf{x}_n)}{1 + \exp(\mathbf{w}^T \mathbf{x}_n)}. \end{aligned}$$

We combine Equations 4.3 and 4.4 to produce a single expression for  $P(T_n = t_n | \mathbf{x}_n, \mathbf{w})$  as follows:

$$P(T_n = t_n | \mathbf{x}_n, \mathbf{w}) = P(T_n = 1 | \mathbf{x}_n, \mathbf{w})^{t_n} P(T_n = 0 | \mathbf{x}_n, \mathbf{w})^{1-t_n},$$

where the observed data ( $t_n$ ) switches the relevant term on and the other off.

Substituting this into Equation 4.2 gives us the likelihood for all  $n$  training points:

$$\begin{aligned} p(\mathbf{t} | \mathbf{X}, \mathbf{w}) &= \prod_{n=1}^N P(T_n = 1 | \mathbf{x}_n, \mathbf{w})^{t_n} P(T_n = 0 | \mathbf{x}_n, \mathbf{w})^{1-t_n} \\ &= \prod_{n=1}^N \left( \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x}_n)} \right)^{t_n} \left( \frac{\exp(-\mathbf{w}^T \mathbf{x}_n)}{1 + \exp(-\mathbf{w}^T \mathbf{x}_n)} \right)^{1-t_n}. \end{aligned} \quad (4.5)$$

**Posterior:** This definition of the likelihood combined with the Gaussian prior we chose earlier are all we need, in theory, to work out the posterior density,  $p(\mathbf{w} | \mathbf{X}, \mathbf{t}, \sigma^2)$ . Once we have the posterior density, we can predict the response (class) of new objects by taking an expectation with respect to this density:

$$P(t_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \mathbf{X}, \mathbf{t}) = \mathbb{E}_{p(\mathbf{w} | \mathbf{X}, \mathbf{t}, \sigma^2)} \left[ \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x}_{\text{new}})} \right].$$

In practice, this is not straightforward. The posterior is not of any standard form. To be able to evaluate it at a particular  $\mathbf{w}$ , we would need to evaluate both the numerator and denominator of Equation 4.1. The numerator is fine – we could evaluate the Gaussian prior density at  $\mathbf{w}$  and the likelihood that we've just defined and multiply the two values together. The denominator is the problem, as we cannot analytically perform the integration required to compute the marginal likelihood:

$$Z^{-1} = p(\mathbf{t} | \mathbf{X}, \sigma^2) = \int p(\mathbf{t} | \mathbf{X}, \mathbf{w}) p(\mathbf{w}) d\mathbf{w}.$$

In other words, we have a function  $g(\mathbf{w}; \mathbf{X}, \mathbf{t}, \sigma^2) = p(\mathbf{t} | \mathbf{X}, \mathbf{w}) p(\mathbf{w} | \sigma^2)$  which we know is proportional to the posterior,  $p(\mathbf{w} | \mathbf{X}, \mathbf{t}, \sigma^2) = Z g(\mathbf{w}; \mathbf{X}, \mathbf{t}, \sigma^2)$ , but we do not know the constant of proportionality,  $Z^{-1}$  (note that this constant is traditionally defined as  $Z_{-1}$  rather than  $Z$ ). We are left with three options:

1. Find the single value of  $\mathbf{w}$  that corresponds to the highest value of the posterior. As  $g(\mathbf{w}; \mathbf{X}, \mathbf{t}, \sigma^2)$  is proportional to the posterior, a maximum of  $g(\mathbf{w}; \mathbf{X}, \mathbf{t}, \sigma^2)$  will also correspond to a maximum of the posterior.  $Z^{-1}$  is not a function of  $\mathbf{w}$ .
2. Approximate  $p(\mathbf{w} | \mathbf{X}, \mathbf{t}, \sigma^2)$  with some other density that we can compute analytically.
3. Sample directly from the posterior  $p(\mathbf{w} | \mathbf{X}, \mathbf{t}, \sigma^2)$ , knowing only  $g(\mathbf{w}; \mathbf{X}, \mathbf{t}, \sigma^2)$ .

The first option is not very Bayesian – we will have to make predictions for new objects based on a single value of  $\mathbf{w}$  and not a density. It is, however, easy to do and this makes it a popular technique. The second option leaves us with a density that is easy

to work with (we can choose any density we like) but if the chosen density is very different from the posterior, our model will not be very reliable. The final option allows us to sample from the posterior (and hence get good approximations to any expectations that we might require) but can be difficult.

These are the three options that are open to us in any problem where we cannot directly compute the posterior density. All three options have good and bad points and the choice of one over another will depend on the specifications (and computational limitations) of the problem at hand. We will now describe each in turn.

### 4.3 A POINT ESTIMATE – THE MAP SOLUTION

In the previous section we showed that, whilst we could not compute the posterior density  $p(\mathbf{w}|\mathbf{X}, \mathbf{t}, \sigma^2)$ , we could compute something proportional to it,  $g(\mathbf{w}; \mathbf{X}, \mathbf{t}, \sigma^2)$ . This is equal to the prior multiplied by the likelihood. The value of  $\mathbf{w}$  that maximises  $g(\mathbf{w}; \mathbf{X}, \mathbf{t}, \sigma^2)$  will also correspond to the value at the maximum of the posterior. This will be the single most likely value of  $\hat{\mathbf{w}}$  (under the posterior) and is a sensible choice if we decide to use a point estimate. Chapter 2 was devoted to finding the value of  $\hat{\mathbf{w}}$  that maximised the likelihood. The idea here is very similar except now we are maximising the likelihood multiplied by the prior. This solution is the maximum a posteriori (MAP) estimate that we first saw in Section 3.8.4 and is common within machine learning.

**Comment 4.1 – The Newton-Raphson method:** The Newton-Raphson method (also known as the Newton method) is a general method for finding points where functions are equal to zero, i.e., finding points where the function  $f(x) = 0$ . Given a current estimate of the zero point,  $x_n$ , we update it by moving to the point where the tangent to the function at  $x_n$  passes through the x-axis. This point can be computed by approximating the gradient as a change in  $f(x)$  divided by a change in  $x$ . Defining  $\partial f(x)/\partial x$  as  $f'(x)$ ,

$$\begin{aligned} f(x_n) &= \frac{f(x_n) - 0}{x_n - x_{n-1}} \\ (x_n - x_{n-1})f'(x_n) &= f(x_n) \\ x_n + 1 &= x_n - \frac{f(x_n)}{f'(x_n)}. \end{aligned}$$

The method can also be used to find minima and maxima, as these are simply points where the gradient passes through zero. Therefore, we simply replace  $f(x)$  with its derivative  $f'(x)$  and  $f'(x)$  with its derivative  $f''(x)$ :

$$x_n + 1 = x_n - \frac{f'(x_n)}{f''(x_n)}.$$

This is readily extendable to functions of a vector – say  $\mathbf{x}$ . In this instance,  $f'(x_n)$  is replaced by the vector of partial derivatives evaluated at  $\mathbf{x}_n$  and  $1/f'(x_n)$  is replaced by the inverse of the Hessian matrix (see Comment 2.6) –  $\partial^2 f(\mathbf{x})/\partial \mathbf{x} \partial \mathbf{x}^T$  – evaluated at  $\mathbf{x} = \mathbf{x}_n$ .

As with finding the maximum likelihood solution, it is easiest to find the value of  $\mathbf{w}$  that maximises  $\log g(\mathbf{w}; \mathbf{X}, \mathbf{t})$  rather than  $g(\mathbf{w}; \mathbf{X}, \mathbf{t})$ :

$$\log g(\mathbf{w}; \mathbf{X}, \mathbf{t}) = \log p(\mathbf{X}|\mathbf{w}) + \log p(\mathbf{w}|\mathbf{P}_n).$$

Unlike the maximum likelihood solution for the linear model, we cannot obtain an exact expression for  $\mathbf{w}$  by differentiating this expression and equating it to zero. Instead, we can use any one of many optimisation algorithms that start with a guess for  $\mathbf{w}$  and

Rogers, Simon, and Mark Girolami. A First Course in Machine Learning. CRC Press LLC, 2016. ProQuest Ebook Central, <http://ebookcentral.proquest.com/lib/uz/redirect.action?docId=4718644>. Created from uz on 2025-04-07 07:45:16.

$$\begin{aligned} \frac{\partial P_n}{\partial \mathbf{w}} &= \frac{\partial(1 + \exp(-\mathbf{w}^T \mathbf{x}_n))^{-1}}{\partial(1 + \exp(-\mathbf{w}^T \mathbf{x}_n))} \frac{\partial(1 + \exp(-\mathbf{w}^T \mathbf{x}_n))}{\partial \mathbf{w}} \\ &= -\frac{1}{(1 + \exp(-\mathbf{w}^T \mathbf{x}_n))^2} \exp(-\mathbf{w}^T \mathbf{x}_n) (-\mathbf{x}_n) \\ &= \frac{\exp(-\mathbf{w}^T \mathbf{x}_n)}{(1 + \exp(-\mathbf{w}^T \mathbf{x}_n))^2} \mathbf{x}_n \\ &= \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x}_n)} \frac{\exp(-\mathbf{w}^T \mathbf{x}_n)}{1 + \exp(-\mathbf{w}^T \mathbf{x}_n)} \mathbf{x}_n \\ &= P_n(1 - P_n)\mathbf{x}_n. \end{aligned} \quad (4.8)$$

**Comment 4.2 – The chain rule:** When taking partial derivatives, it is often convenient to use the chain rule. The chain rule states that

$$\frac{\partial f(\mathbf{g}(\mathbf{w}))}{\partial \mathbf{w}} = \frac{\partial f(\mathbf{g}(\mathbf{w}))}{\partial \mathbf{g}(\mathbf{w})} \frac{\partial \mathbf{g}(\mathbf{w})}{\partial \mathbf{w}}.$$

As an example, let

$$f(\mathbf{w}) = t_n \log P_n$$

where

$$P_n = \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x}_n)}.$$

To compute  $\frac{\partial f(\mathbf{w})}{\partial \mathbf{w}}$ , we can use the chain rule as follows:

$$\frac{\partial f(\mathbf{w})}{\partial \mathbf{w}} = \frac{\partial f(\mathbf{w})}{\partial P_n} \frac{\partial P_n}{\partial \mathbf{w}} = t_n \frac{\partial P_n}{\partial \mathbf{w}}.$$

Substituting Equation 4.8 into Equation 4.7 gives us the required vector of partial derivatives:

$$\begin{aligned} \frac{\partial \log g(\mathbf{w}; \mathbf{X}, \mathbf{t})}{\partial \mathbf{w}} &= -\frac{1}{\sigma^2} \mathbf{w} + \sum_{n=1}^N (\mathbf{x}_n t_n (1 - P_n) - \mathbf{x}_n (1 - t_n) P_n) \\ &= -\frac{1}{\sigma^2} \mathbf{w} + \sum_{n=1}^N \mathbf{x}_n (t_n - t_n P_n - P_n + t_n P_n) \\ &= -\frac{1}{\sigma^2} \mathbf{w} + \sum_{n=1}^N \mathbf{x}_n (t_n - P_n). \end{aligned} \quad (4.9)$$

To compute the Hessian matrix of second derivatives, we differentiate this again with respect to  $\mathbf{w}^T$ . Noting that  $\frac{\partial P_n}{\partial \mathbf{w}^T} = \left( \frac{\partial P_n}{\partial \mathbf{w}} \right)^T$ , we obtain the following expression:

$$\begin{aligned} \frac{\partial^2 \log g(\mathbf{w}; \mathbf{X}, \mathbf{t})}{\partial \mathbf{w} \partial \mathbf{w}^T} &= -\frac{1}{\sigma^2} \mathbf{I} - \sum_{n=1}^N \mathbf{x}_n \frac{\partial P_n}{\partial \mathbf{w}^T} \\ &= -\frac{1}{\sigma^2} \mathbf{I} - \sum_{n=1}^N \mathbf{x}_n \mathbf{x}_n^T P_n (1 - P_n). \end{aligned} \quad (4.10)$$

One thing to notice from the Hessian is that, because  $0 \leq P_n \leq 1$ , it will be negative definite for any set of  $\mathbf{x}_n$  and for any  $\mathbf{w}$  (see Section 2.8.3). Therefore, there can only be one optimum and it must be a maximum. Whatever value of  $\mathbf{w}$  the Newton-

then keep updating it in such a way that  $g(\mathbf{w}; \mathbf{X}, \mathbf{t})$  increases until a maximum is reached. The Newton-Raphson procedure (see Comment 4.1) is one such method that updates  $\mathbf{w}$  using the following equation:

$$\mathbf{w} = \mathbf{w} - \left( \frac{\partial^2 \log g(\mathbf{w}; \mathbf{X}, \mathbf{t})}{\partial \mathbf{w} \partial \mathbf{w}^T} \right)^{-1} \frac{\partial \log g(\mathbf{w}; \mathbf{X}, \mathbf{t})}{\partial \mathbf{w}}. \quad (4.6)$$

The new version ( $\mathbf{w}'$ ) of  $\mathbf{w}$  is calculated by subtracting the inverse of the Hessian (see Comment 2.6) multiplied by the vector of partial derivatives. For any starting value of  $\mathbf{w}$ , this iterative procedure will update  $\mathbf{w}$  until it reaches a point where the gradient is zero. To check that the point we have converged to corresponds to a maximum, we can check the Hessian to ensure that it is negative definite, just as we did for maximum likelihood in Section 2.8.3.

In order to compute the vector of first derivatives, we first expand our expression for  $\log g(\mathbf{w}; \mathbf{X}, \mathbf{t})$  using Equations 4.2 and 4.5:

$$\begin{aligned} \log g(\mathbf{w}; \mathbf{X}, \mathbf{t}) &= \sum_{n=1}^N \log P(T_n = t_n | \mathbf{x}_n, \mathbf{w}) + \log p(\mathbf{w} | \sigma^2) \\ &= \sum_{n=1}^N \log \left[ \left( \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x}_n)} \right)^{t_n} \frac{\exp(-\mathbf{w}^T \mathbf{x}_n)}{1 + \exp(-\mathbf{w}^T \mathbf{x}_n)} \right]^{1-t_n} + \log p(\mathbf{w} | \sigma^2). \end{aligned}$$

To stop this expression becoming too complicated, we will use the following shorthand:

$$P_n = P(T_n = 1 | \mathbf{x}_n, \mathbf{w}) = \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x}_n)}.$$

Therefore, assuming that  $\mathbf{w}$  is  $D$ -dimensional, we have the following expression:

$$\begin{aligned} \log g(\mathbf{w}; \mathbf{X}, \mathbf{t}) &= \log p(\mathbf{w} | \sigma^2) + \sum_{n=1}^N \log P_n^t + \log(1 - P_n)^{1-t_n} \\ &= -\frac{D}{2} \log 2\pi - D \log \sigma - \frac{1}{2\sigma^2} \mathbf{w}^T \mathbf{w} + \sum_{n=1}^N t_n \log P_n + (1 - t_n) \log(1 - P_n), \end{aligned}$$

where the first three terms are the log of the (Gaussian) prior. To find the vector of partial derivatives, we can use the chain rule (see Comment 4.2) to give an expression in terms of the partial derivatives of  $P_n$ :

$$\begin{aligned} \frac{\partial \log g(\mathbf{w}; \mathbf{X}, \mathbf{t})}{\partial \mathbf{w}} &= -\frac{1}{\sigma^2} \mathbf{w} + \sum_{n=1}^N \left( \frac{t_n}{P_n} \frac{\partial P_n}{\partial \mathbf{w}} - \frac{1 - t_n}{1 - P_n} \frac{\partial(1 - P_n)}{\partial \mathbf{w}} \right) \\ &= -\frac{1}{\sigma^2} \mathbf{w} + \sum_{n=1}^N \left( \frac{t_n}{P_n} \frac{\partial P_n}{\partial \mathbf{w}} - \frac{1 - t_n}{1 - P_n} \frac{\partial P_n}{\partial \mathbf{w}} \right), \end{aligned} \quad (4.7)$$

where we have used the chain rule a second time to turn  $\frac{\partial(1 - P_n)}{\partial \mathbf{w}}$  into  $-\frac{\partial P_n}{\partial \mathbf{w}}$ :

$$\begin{aligned} \frac{\partial(1 - P_n)}{\partial \mathbf{w}} &= \frac{\partial(1 - P_n)}{\partial P_n} \frac{\partial P_n}{\partial \mathbf{w}} \\ &= -\frac{\partial P_n}{\partial \mathbf{w}}. \end{aligned}$$

To calculate  $\frac{\partial P_n}{\partial \mathbf{w}}$ , we can use the chain rule once more:

Rogers, Simon, and Mark Girolami. A First Course in Machine Learning. CRC Press LLC, 2016. ProQuest Ebook Central, <http://ebookcentral.proquest.com/lib/uz/redirect.action?docId=4718644>. Created from uz on 2025-04-07 07:45:16.

Rogers, Simon, and Mark Girolami. A First Course in Machine Learning. CRC Press LLC, 2016. ProQuest Ebook Central, <http://ebookcentral.proquest.com/lib/uz/redirect.action?docId=4718644>. Created from uz on 2025-04-07 07:45:16.

Raphson procedure converges to must correspond to the highest value of the posterior density. This is a consequence of the choice of prior and likelihood function, and changing either may result in a harder posterior density to optimise.

We now have everything we need to perform the Newton-Raphson procedure and find a potential optimal value of  $\mathbf{w}$ . Starting with  $\mathbf{w} = [0, 0]^T$  and setting  $\sigma^2 = 10$ , the procedure converges (the change in  $\mathbf{w}$  becomes insignificant) after only nine iterations (MATLAB script: `logmap.m`). The evolution of the two components of  $\mathbf{w}$  over this period can be seen in Figure 4.3. Following the previous chapters, we will call the value of  $\mathbf{w}$  that corresponds to the maximum  $\hat{\mathbf{w}}$ .

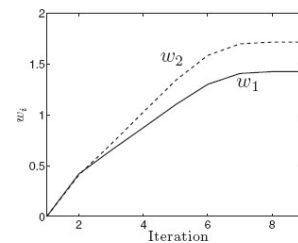


FIGURE 4.3 Evolution of the components of  $\mathbf{w}$  throughout the Newton-Raphson procedure to find the  $\mathbf{w}$  corresponding to the maximum of the posterior density.

Using  $\hat{\mathbf{w}}$ , we can compute the probability that the response equals 1 for any  $\mathbf{x}$ . In particular, if we observe  $\mathbf{x}_{\text{new}}$ , a new set of attributes, the probability that it should be given a response of 1 (it belongs to the square class) is given by

$$P(T_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \hat{\mathbf{w}}) = \frac{1}{1 + \exp(-\hat{\mathbf{w}}^T \mathbf{x}_{\text{new}})}. \quad (4.11)$$

Given that there are two possible responses (or classes) for this new object, a sensible strategy might be to assign it to the square class ( $T_{\text{new}} = 1$ ) if the probability is greater than 0.5 and to the circle class ( $T_{\text{new}} = 0$ ) otherwise. In this case, the set of  $\mathbf{x}$  values that correspond to  $P(T = 1 | \mathbf{x}, \hat{\mathbf{w}}) = 0.5$  will form a line that can be thought of as a **decision boundary** – points on one side of the line will belong to one class, and points on the other side to the other class. To plot the decision boundary, we make use of the fact that  $P(T = 1 | \mathbf{x}, \hat{\mathbf{w}}) = 0.5$  implies that  $\hat{\mathbf{w}}^T \mathbf{x} = 0$  (see Exercise 4.5). If we expand this expression, we can obtain the decision boundary as a function of  $x_1$  and  $x_2$ :

$$\begin{aligned} 0 &= \hat{\mathbf{w}}^T \mathbf{x} \\ &= \hat{w}_1 x_1 + \hat{w}_2 x_2 \\ \hat{w}_2 x_2 &= -\hat{w}_1 x_1 \\ x_2 &= -\frac{\hat{w}_1 x_1}{\hat{w}_2}, \end{aligned}$$

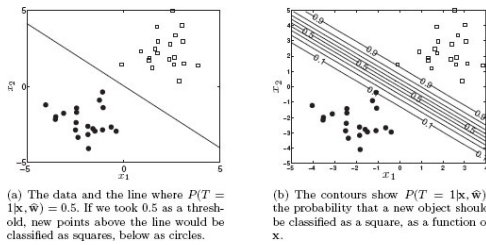


FIGURE 4.4 Inferred function in the binary response example.

which is plotted in Figure 4.4(a). If we want to split the two classes with a straight line, this seems like quite a reasonable choice. In Figure 4.4(b) we plot contours of  $P(T = 1|x, \hat{w})$  as a function of  $x$  (MATLAB script: `logmap.m`). Close to the squares the probability is 1 (the squares are objects for which  $t_n = 1$ ) and close to the circles it is 0. Between the two groups of data, the probability is around 0.5, reflecting the fact that objects here would be equidistant from both groups.

The outcome of this optimisation is that we have a model with which we can make predictions. The model is based on a point estimate,  $\hat{w}$ , of the parameters that we have obtained by finding the value of  $w$  that corresponds to a maximum of the posterior,  $p(w|x, t, \sigma^2)$ . This MAP solution is common in machine learning because it is reasonably easy to find  $\hat{w}$  in this way. One could follow the steps described above for any prior and likelihood combination and find an optimum value. The optimisations will not always be as well behaved as this – in some problems, the posterior might have several maxima (and maybe even some minima). It would be difficult to know if the maximum we had found using Newton–Raphson was the **global optimum**.

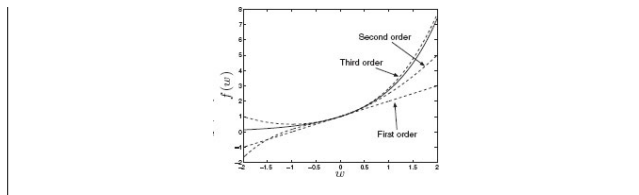
In Chapter 3 we have already seen the advantage of maintaining a density over  $w$  rather than collapsing onto a point estimate. With this in mind, we will now move on to our second option when faced with a posterior we cannot compute exactly – finding a density that approximates  $p(w|x, t, \sigma^2)$ .

#### 4.4 THE LAPLACE APPROXIMATION

There are various approximation methods used within machine learning to replace tricky posterior densities with approximations that are easier to handle. The most popular is the Laplace approximation.<sup>1</sup> The idea is to approximate the density of interest with a Gaussian. Given the ease with which we can manipulate Gaussians, this seems to be a sensible choice – the expectations required to make predictions are likely to be easy to calculate given a Gaussian posterior. However, we should always bear in mind that our predictions will then only be as good as our approximation. If our true posterior is not very Gaussian, our predictions will be easy to compute but not very useful.

The Gaussian density is defined by its mean and (co)variance. Using a Gaussian to approximate another density amounts to choosing suitable values for these parameters. To motivate the choices of parameters made by the Laplace approximation, imagine that, rather than having two parameters, our model has only one –  $w$  – and that we know  $\hat{w}$  – the value corresponding to the highest value of the posterior. Our first step is to approximate  $\log g(w; X, t, \sigma^2)$  using a Taylor expansion (see Comment 4.3) around the maximum,  $\hat{w}$ :

Rogers, Simon, and Mark Girolami. A First Course in Machine Learning. CRC Press LLC, 2016. ProQuest Ebook Central, <http://ebookcentral.proquest.com/lib/uz/videtal.action?docID=4718644>. Created from uz on 2025-04-07 07:45:16.



This idea is easily extended to multivariate densities. In particular, the Laplace approximation to our true posterior  $p(w|x, t, \sigma^2)$  is

$$p(w|x, t, \sigma^2) \approx N(\mu, \Sigma),$$

where  $\mu$  is set to  $\hat{w}$ , and  $\Sigma$  is the negative of the inverse Hessian:

$$\mu = \hat{w}, \quad \Sigma^{-1} = - \left( \frac{\partial^2 \log g(w; X, t, \sigma^2)}{\partial w \partial w^T} \right)_{\hat{w}}. \quad (4.13)$$

##### 4.4.1 Laplace approximation example: Approximating a gamma density

Before we look at what this approximation looks like in the binary response example, it is useful to look at an example where we know the true density (see also Exercises 4.1, 4.2 and 4.3) (MATLAB script: `lapexample.m`). This will allow us to see how good or bad the approximation is. The following is the gamma density for a random variable  $Y$ :

$$p(y|\alpha, \beta) = \frac{\beta^\alpha}{\Gamma(\alpha)} y^{\alpha-1} \exp\{-\beta y\}. \quad (4.14)$$

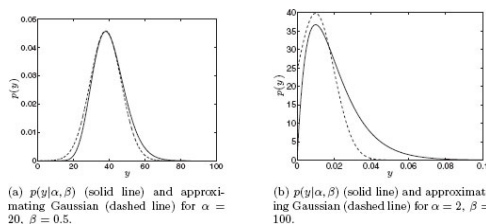


FIGURE 4.5 Examples of the Laplace approximation to the gamma density function given in Equation 4.14.

We will investigate how good the Laplace approximation is to this density. The gamma density has an analytic expression for its mode which means we do not need to go through an optimisation procedure similar to that in the last section. The mode,  $\hat{y}$ , is defined as

$$\hat{y} = \frac{\alpha-1}{\beta}.$$

$$\begin{aligned} \log g(w; X, t, \sigma^2) &\approx \log g(\hat{w}; X, t, \sigma^2) + \frac{\partial \log g(w; X, t, \sigma^2)}{\partial w} \bigg|_{\hat{w}} (w - \hat{w}) \\ &\quad + \frac{\partial^2 \log g(w; X, t, \sigma^2)}{\partial w^2} \bigg|_{\hat{w}} \frac{(w - \hat{w})^2}{2} + \dots \end{aligned}$$

The second term is the first derivative (i.e. the gradient) evaluated at the maximum point and must therefore be zero. Discarding this, and ignoring terms of third-order and above, we are left with the following expression:

$$\log g(w; X, t, \sigma^2) \approx \log g(\hat{w}; X, t, \sigma^2) - \frac{1}{2} (w - \hat{w})^2, \quad (4.12)$$

where  $v$  is the negative of the second derivative of  $\log g(w; X, t, \sigma^2)$  evaluated at  $w = \hat{w}$ :

$$v = - \frac{\partial^2 \log g(w; X, t, \sigma^2)}{\partial w^2} \bigg|_{\hat{w}}.$$

Now, the Gaussian density is defined as

$$\frac{1}{\sqrt{2\pi}\sigma} \exp\left\{-\frac{1}{2\sigma^2}(w - \mu)^2\right\},$$

the log of which is equal to

$$\log(K) - \frac{1}{2\sigma^2}(w - \mu)^2,$$

where  $K$  is the normalising constant. This looks very similar to Equation 4.12 with  $\mu = \hat{w}$  and  $\sigma^2 = 1/v$ . This is the Laplace approximation – we approximate the posterior with a Gaussian that has its mean at the posterior mode ( $\hat{w}$ ) and has variance inversely proportional to the curvature of the posterior (its second derivative) at its mode.

**Comment 4.3 – Taylor expansions:** The Taylor expansion is a way of approximating a function. The approximation is always made ‘about’ some value – the approximation will tend to diverge from the true function as we move away from that value. The definition of the Taylor series of  $f(w)$  about  $\hat{w}$  is

$$f(w) = \sum_{n=0}^{\infty} \frac{(w - \hat{w})^n}{n!} \frac{\partial^n f(w)}{\partial w^n} \bigg|_{\hat{w}}$$

where  $\frac{\partial^n f(w)}{\partial w^n} \bigg|_{\hat{w}}$  is the  $n$ th derivative of  $f(w)$  with respect to  $w$ , evaluated at  $\hat{w}$ . When  $n = 0$ , this derivative is simply the function  $f(w)$ . If we only compute a finite number of terms, we will have an approximation to the function. A first-order approximation would just include terms  $n = 0$  and  $n = 1$  – an  $n$ th-order approximation includes all terms up to and including term  $n$ . For example, we can approximate  $f(w) = \exp(w)$  at  $\hat{w} = 0$ :

$$\exp(w) = \exp(\hat{w}) + \frac{w}{1} \exp(\hat{w}) + \frac{w^2}{2} \exp(\hat{w}) + \dots$$

Now,  $\exp(\hat{w}) = 1$ , so

$$\exp(w) = 1 + \frac{w}{1} + \frac{w^2}{2} + \frac{w^3}{3} + \dots$$

The approximation will get better and better as we add more and more terms. This can be seen in the figure on the right.

Rogers, Simon, and Mark Girolami. A First Course in Machine Learning. CRC Press LLC, 2016. ProQuest Ebook Central, <http://ebookcentral.proquest.com/lib/uz/videtal.action?docID=4718644>. Created from uz on 2025-04-07 07:45:16.

The Laplace approximation to  $p(y|\alpha, \beta)$  takes the form of a Gaussian:

$$p(y|\alpha, \beta) \approx N(\mu, \sigma^2).$$

The mean  $\mu$  will be equal to the mode of  $p(y|\alpha, \beta)$ , which we’ve already defined. To find the variance,  $\sigma^2$ , of the approximating Gaussian, we need to find the second derivative of  $\log p(y|\alpha, \beta)$  with respect to  $y$ . This is computed as follows:

$$\begin{aligned} \log p(y|\alpha, \beta) &= \alpha \log \beta - \log \Gamma(\alpha) + (\alpha - 1) \log y - \beta y \\ \frac{\partial \log p(y|\alpha, \beta)}{\partial y} &= \frac{\alpha - 1}{y} - \beta \\ \frac{\partial^2 \log p(y|\alpha, \beta)}{\partial y^2} &= -\frac{\alpha - 1}{y^2}. \end{aligned}$$

$\sigma^2$  will be equal to the negative inverse of this quantity evaluated at  $y = \hat{y}$ . In particular

$$\sigma^2 = \frac{\hat{y}^2}{\alpha - 1} = \frac{\alpha - 1}{\beta^2}.$$

In Figure 4.5 we can see two examples of  $p(y|\alpha, \beta)$  and the corresponding Laplace approximation. In the first,  $p(y|\alpha, \beta)$  looks rather like a Gaussian and the approximation is pretty good. In the second,  $p(y|\alpha, \beta)$  does not look very much like a Gaussian and the approximation is not accurate. In both cases the approximation gets worse as we move away from the mode. This is because the approximation is based on the characteristics of the function at the mode. We will see this property again as we return to the binary response model.

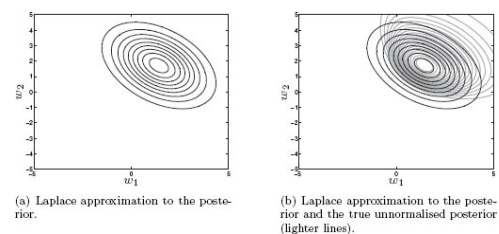


FIGURE 4.6 The Laplace approximation for the binary problem.

##### 4.4.2 Laplace approximation for the binary response model

Returning to our binary response model, we had to compute both the mode,  $\hat{w}$  and the Hessian for the Newton–Raphson procedure. We therefore already have everything we need for the Laplace approximation to the posterior  $p(w|x, t, \sigma^2)$ . In Figure 4.6(a) we can see the approximate posterior and in Figure 4.6(b) we can see the same approximation on top of  $g(w; X, t)$ , the unnormalised posterior. As for the gamma example in the previous section, the shape of the approximation is pretty good around the mode but diverges considerably from the true posterior as we move away from the mode. This is to be expected – the Laplace approximation only matches the shape (curvature) at the mode. We can also sample values of  $w$  from the approximate posterior and look at the decision boundaries that they correspond to. Twenty such boundaries are plotted in Figure 4.7(a). There appears to be a lot of variability in these boundaries, although all of them seem to split the classes reasonably well.

The final step is to use the approximate posterior to compute predictions. We now have a density over  $\mathbf{w}$  rather than a single value and we know, from Chapter 3, that we compute a prediction by averaging over this density. In particular, we should be calculating the expected value of  $P(T_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \mathbf{w})$  with respect to the approximate posterior over  $\mathbf{w}$  (which we've denoted as  $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ ):

$$P(T_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \mathbf{X}, \mathbf{t}, \sigma^2) = \mathbb{E}_{\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})} \{P(T_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \mathbf{w})\}.$$

Unfortunately, we cannot compute the integral over  $\mathbf{w}$  required in this expectation. This might suggest that our choice of approximation was not sensible – we still cannot make predictions. However, we can easily sample from  $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$  and so (see Equation 2.23) we can approximate the expectation with

$$P(T_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \mathbf{X}, \mathbf{t}, \sigma^2) \approx \frac{1}{N_s} \sum_{s=1}^{N_s} \frac{1}{1 + \exp(-\mathbf{w}_s^T \mathbf{x}_{\text{new}})}, \quad (4.15)$$

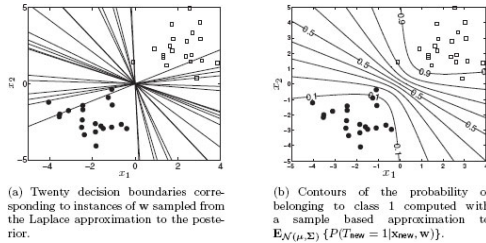


FIGURE 4.7 Decision boundaries sampled from the Laplace approximation and the predictive probability contours.

where  $\mathbf{w}_s$  is the  $s$ th of  $N_s$  samples drawn from the approximate posterior. Using  $N_s = 1000$ , the contours of  $P(T_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \mathbf{X}, \mathbf{t}, \sigma^2)$  can be seen in Figure 4.7(b) (MATLAB script: `log1ap.m`). Compare this with Figure 4.4(b). There is a big difference – the contours are no longer straight lines. Averaging over the posterior density for  $\mathbf{w}$  has had the effect of smudging the decision boundaries. The probabilities are now closer to 0.5 in all areas except those very close to the data objects. The model based on the point estimate, shown in Figure 4.4(b), could be said to be overconfident – take  $x_1 = -3, x_2 = 5$  as an example. According to the predictions produced by the point estimate (Figure 4.4(b)), an object with these attributes would have a probability of approximately 1 of being a square despite the fact that it is quite distant from the other square objects. Compare this with the probability of approximately 0.6 given by the expectation with respect to the Laplace approximation to the posterior (Figure 4.7(b)). This value seems much more reasonable. Another way to understand the uncertainty that should be present in areas like this is to look at Figure 4.7(a) – there is very large variability in the possible decision boundaries at  $x_1 = -3, x_2 = 5$ . Some of these boundaries would classify this object as a square, some as a circle – the probability that it is a square, given the data that we have seen, is not 1.

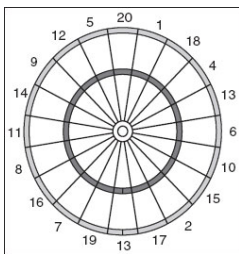


FIGURE 4.8 A dartboard.

Assume that there exists some probability density function defined over the position that the dart will land. In other words, when a player is aiming for, say, double twenty, the position at which the dart will land could be considered as an instance of some random variable. We will use the vector  $\mathbf{y}$  to describe this position and therefore the density will look something like  $p(\mathbf{y} | \Delta)$ . A ought to depend (at least to some extent) on where the player is aiming. The extent to which this dependence exists depends on the player's skill. For a professional trying to hit double twenty, we might expect the density to be tightly concentrated around the double twenty area. For a poor player, aiming might make very little difference to where the dart ends up. So,  $\Delta$  depends on the skill of the player and the strength of their technique – making  $p(\mathbf{y} | \Delta)$  very hard to define.

At this point it would be easy to give up. But, taking a step backwards, we are not directly interested in  $p(\mathbf{y} | \Delta)$ , just the probability that the player throws a double 20. Do we need to be able to write down an analytic expression for  $p(\mathbf{y} | \Delta)$  to work this out? Before we answer this, let us satisfy ourselves that we could work it out if we could write down  $p(\mathbf{y} | \Delta)$ . Define the random variable  $T = f(\mathbf{y})$  where  $f(\mathbf{y})$  is 1 if  $\mathbf{y}$  is inside the double twenty region and zero otherwise.  $T$  depends on  $\mathbf{y}$  and hence depends on  $\Delta$ . So, we're interested in the following probability:  $P(T = 1 | \Delta)$ . This is nothing more than an expectation. In particular, it looks rather like the expectations we had to compute for the binary response model in the previous section:

$$P(T = 1 | \Delta) = \mathbb{E}_{p(\mathbf{y} | \Delta)} \{f(\mathbf{y})\} = \int f(\mathbf{y}) p(\mathbf{y} | \Delta) d\mathbf{y}. \quad (4.16)$$

In theory, if we could write down  $p(\mathbf{y} | \Delta)$ , we could work this out. However, we have also seen that we can compute quantities like this with a sample-based approximation. In particular, if  $\mathbf{y}_s$  is the  $s$ th of  $N_s$  samples from  $p(\mathbf{y} | \Delta)$ , our approximation would look like:

$$P(T = 1 | \Delta) \approx \frac{1}{N_s} \sum_{s=1}^{N_s} f(\mathbf{y}_s)$$

So, we do not need to be able to write down  $p(\mathbf{y} | \Delta)$  to be able to compute  $P(T = 1 | \Delta)$  as long as we can sample from it. Fortunately, sampling from  $p(\mathbf{y} | \Delta)$  is pretty easy – we get our player, some darts and a board and we ask the player to aim for double twenty. The position of each dart thrown is a sample from  $p(\mathbf{y} | \Delta)$ . If we record  $\mathbf{y}_s$  for each of  $N_s$  throws, we can compute the sample-based approximation given in Equation 4.16. In fact, in this case it works out as just the proportion of times the player throws a double twenty).

In this section we have seen again that we should be wary of using point estimates. The Laplace approximation shown here can be used to approximate any density (over real-valued random variables) for which we can find the mode and compute the second derivative. The approach assumes that the posterior can be reasonably approximated by a Gaussian, something that is not always the case (see Figure 4.5). In our binary response model, the approximation did not allow us to compute the expectation necessary for making predictions exactly. However, the ease with which we can sample from a Gaussian meant that it was straightforward to obtain a sample-based approximation to the expectation. In the next section, we will extend this idea through the introduction of a technique that will enable us to sample directly from  $p(\mathbf{w} | \mathbf{X}, \mathbf{t}, \sigma^2)$  despite the fact that we cannot compute the normalisation constant. The ability to generate these samples will allow us to use a sample-based approximation to the expectation without having to approximate the posterior.

## 4.5 SAMPLING TECHNIQUES

The Laplace approximation in the previous section provided us with a method for approximating the posterior density  $p(\mathbf{w} | \mathbf{X}, \mathbf{t}, \sigma^2)$ . Our interest in the posterior density is primarily to allow us to take all the uncertainty in  $\mathbf{w}$  into account when making predictions. We do this by averaging over all potential values of  $\mathbf{w}$  through the following expectation:

$$P(T_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \mathbf{X}, \mathbf{t}, \sigma^2) = \mathbb{E}_{p(\mathbf{w} | \mathbf{X}, \mathbf{t}, \sigma^2)} \{P(T_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \mathbf{w})\}.$$

Even substituting our approximation to the posterior into this expression, we could not analytically compute the integral required in this expectation. Fortunately, it was easy to sample from the Gaussian approximation, enabling us to use the sample based approximation given in Equation 4.15. In this instance, the benefit of making the approximation was that it enabled us to easily generate samples. In this section, we will look at a technique that enables us to cut out the approximation step and sample directly from the posterior. A set of samples from the true posterior generated in this way could be substituted directly into Equation 4.15 to compute the desired predictive probability,  $P(T_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \mathbf{X}, \mathbf{t}, \sigma^2)$ . We're going to introduce a popular sampling technique known as the **Metropolis–Hastings** algorithm. However, before we go into this, it is perhaps useful to get more comfortable with the idea of sampling through a less abstract example.

### 4.5.1 Playing darts

In the game of darts, players take turns to throw three darts at a board like that shown in Figure 4.8. The darts are sharp and embed themselves into the board. The player receives a certain number of points for each dart, depending on where the dart lands. The scores from the three darts are added together and subtracted from the player's current total. Each player starts the game with the same total (normally 501) and the winner is the player who gets to zero first. The majority of the board is split into 20 segments and if the dart lands in the white parts of these segments, the score is equal to the number shown around the edge. If the dart lands in one of the shaded areas, the score is either double (lighter, outer shaded area) or triple (darker, inner area) the segment score. The circle in the centre of the board is known as the bull's-eye (50 points) and the circle around this as the bull (25 points). There is one slight complication to the rules – the player must get to zero with a double. So, for example, if a player currently has a total of 40, they could win by throwing a double 20 (the lightly shaded area just below the '20' label) or a single 20 (anywhere in the white bits of the '20' segment) followed by a double 10, etc. We will assume that the player does indeed need to score 40 to win, and has only one dart left with which to do it. In other words, they need to hit the double 20 – what is the probability that they will succeed?

We can explicitly relate this procedure to our binary response model. Firstly, the quantity of interest in the darts case,  $P(T = 1 | \Delta)$  is analogous to the predictive probability in the binary response model:  $P(T_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \mathbf{X}, \mathbf{t}, \sigma^2)$ . In both cases to compute this quantity, we must take an expectation with respect to some density: our darts distribution  $p(\mathbf{y} | \Delta)$  is analogous to  $p(\mathbf{w} | \mathbf{X}, \mathbf{t}, \sigma^2)$  – the posterior density over our parameters. In the darts case, we approximated this expectation by drawing samples directly from the posterior (despite the fact that we couldn't write it down). In the binary response case, we approximated the posterior with something we could sample from and then sampled. We will now see how we can sample directly from  $p(\mathbf{w} | \mathbf{X}, \mathbf{t}, \sigma^2)$  (see Exercise 4.4).

### 4.5.2 The Metropolis–Hastings algorithm

In this section, we will introduce the Metropolis–Hastings<sup>2</sup> (MH) algorithm. Rather than go into too much detail we will introduce it as a recipe, describing the steps involved without proving why they work. References to further reading are provided at the end of the chapter.

Recall that we are attempting to sample from  $p(\mathbf{w} | \mathbf{X}, \mathbf{t}, \sigma^2)$  so that we can approximate the following expectation:

$$P(T_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \mathbf{X}, \mathbf{t}, \sigma^2) = \mathbb{E}_{p(\mathbf{w} | \mathbf{X}, \mathbf{t}, \sigma^2)} \{P(T_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \mathbf{w})\} = \int P(T_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \mathbf{w}) p(\mathbf{w} | \mathbf{X}, \mathbf{t}, \sigma^2) d\mathbf{w},$$

with:

$$P(T = 1 | \mathbf{x}_{\text{new}}, \mathbf{X}, \mathbf{t}, \sigma^2) \approx \frac{1}{N_s} \sum_{s=1}^{N_s} P(T_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \mathbf{w}_s).$$

Metropolis–Hastings generates a sequence of samples  $\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_{s-1}, \mathbf{w}_s, \dots, \mathbf{w}_{N_s}$ . Generating a sample (say  $\mathbf{w}_s$ ) consists of two steps. In the first step, we need to *propose* a new sample – a candidate for  $\mathbf{w}_s$ . This is performed by proposing a movement from the previous sample ( $\mathbf{w}_{s-1}$ ). Secondly, the proposed sample is tested to see whether or not it should be accepted. If accepted, it becomes our new sample  $\mathbf{w}_s$ . If it is not accepted, our new sample is set to be equal to the previous one,  $\mathbf{w}_s = \mathbf{w}_{s-1}$ . This is continued until we have collected what we believe to be enough samples.

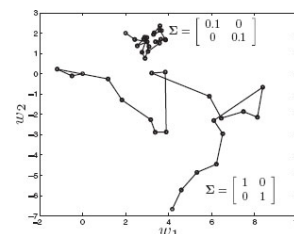


FIGURE 4.9 Two examples of random walks where the distribution over the next location is a Gaussian centred at the current location. The two walks have different covariance matrices, shown in the plot.

Now, if our proposal is based on a movement from the previous sample, what do we do for our first sample  $\mathbf{w}_1$ ? It turns out that it doesn't matter where we start –  $\mathbf{w}_1$  can be anything. As long as we sample for long enough, our sampler is guaranteed to converge to the distribution of interest. So, we can pluck a  $\mathbf{w}_1$  from anywhere (sampling it from the prior would probably be a sensible choice), set the Metropolis–Hastings algorithm off, wait for it to converge to the correct distribution and then harvest as many samples as we need. A word of caution: the sampler is guaranteed to converge *in theory*. In practice, it is important to use one (or ideally more) of the methods available to test convergence before we start harvesting samples. We will now look at the proposal and acceptance steps in more detail.

**Proposing a new sample** Assume that we have already sampled  $s - 1$  values using the MH scheme. We will propose a sample based on a movement from  $\mathbf{w}_{s-1}$ . Calling our proposed sample  $\tilde{\mathbf{w}}_s$  (we can only call it  $\mathbf{w}_s$  once it has been accepted), we need to define a density:

$$p(\tilde{\mathbf{w}}_s | \mathbf{w}_{s-1})$$

This density does not have to have any connection with the posterior  $p(\mathbf{w} | \mathbf{X}, \mathbf{t}, \sigma^2)$  from which we're trying to sample. We are free to define it as we please. In practice, the choice will have an impact on how long it will take the MH sampler to converge. A common choice is to use a Gaussian centred on the current sample,  $\mathbf{w}_{s-1}$ :

$$p(\tilde{\mathbf{w}}_s | \mathbf{w}_{s-1}, \Sigma) = \mathcal{N}(\tilde{\mathbf{w}}_s - \mathbf{w}_{s-1}, \Sigma)$$

Sampling a sequence of values like this creates what is known as a **random walk**. In Figure 4.9 we show two such walks

(MATLAB script: `randwalks.m`). One starts from  $\mathbf{w}_1 = [0, 0]^T$  and has covariance  $\Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$  whilst the other starts from  $\mathbf{w}_1 = [2, 2]^T$  and has covariance  $\Sigma = \begin{bmatrix} 0.1 & 0 \\ 0 & 0.1 \end{bmatrix}$ . The latter walk moves less distance in each step due to the smaller diagonal (variance) elements in the covariance matrix. As we have already mentioned, the Gaussian is a popular choice for the proposal density. One reason is the ease with which we can sample from it – choosing a proposal distribution that was hard to sample from would make things unnecessarily complicated. Another reason is that it is symmetric: moving to  $\tilde{\mathbf{w}}_s$  from  $\mathbf{w}_{s-1}$  is just as likely as moving from  $\tilde{\mathbf{w}}_s$  to  $\mathbf{w}_{s-1}$ :

$$p(\tilde{\mathbf{w}}_s | \mathbf{w}_{s-1}, \Sigma) = p(\mathbf{w}_{s-1} | \tilde{\mathbf{w}}_s, \Sigma)$$

We will see the advantage of this as we move on to the acceptance step.

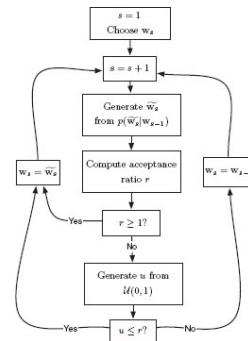


FIGURE 4.10 The Metropolis–Hastings algorithm.

**Accepting or rejecting** We now have  $\tilde{\mathbf{w}}_s$ , a candidate for  $\mathbf{w}_s$ . We must now decide whether we should accept it or reject it.

To do this, we compute the following ratio:

$$r = \frac{p(\tilde{\mathbf{w}}_s | \mathbf{X}, \sigma^2) p(\mathbf{w}_{s-1} | \tilde{\mathbf{w}}_s)}{p(\mathbf{w}_s | \mathbf{X}, \sigma^2) p(\tilde{\mathbf{w}}_s | \mathbf{w}_{s-1})}. \quad (4.17)$$

This is the ratio of the posterior density at the *proposed* sample to that at the old sample multiplied by the ratio of the proposal densities. The symmetry of the Gaussian proposal distribution discussed above allows us to ignore this last term, as it is always equal to 1. The first term is the ratio of posterior densities evaluated at the two different parameter values. We cannot compute the densities exactly because we cannot normalise them. However, because we are interested in a ratio, the normalisation constants cancel. So, we can substitute the ratio of posteriors with the ratio of the priors multiplied by the ratio of likelihoods. This leads us to the following expression:

$$r = \frac{g(\tilde{\mathbf{w}}_s | \mathbf{X}, \sigma^2)}{g(\mathbf{w}_s | \mathbf{X}, \sigma^2)} = \frac{p(\tilde{\mathbf{y}} | \tilde{\mathbf{w}}_s)}{p(\mathbf{y} | \mathbf{w}_s)} \frac{p(\tilde{\mathbf{w}}_s)}{p(\mathbf{w}_s)}.$$

This ratio will always be positive as the density functions are always positive. If it is one or greater, we accept the sample ( $\mathbf{w}_s = \tilde{\mathbf{w}}_s$ ). If  $r$  is less than 1, we accept the sample with probability equal to  $r$ . In other words, if we propose a set of parameters that corresponds to a higher value of the posterior density than  $\mathbf{w}_{s-1}$ , we always accept it ( $r > 1$ ). If we propose a set that corresponds to a lower value of posterior density, we accept it sometimes, but not always. The algorithm is depicted in Figure 4.10. Notice that we have described the accept/reject step in more detail. If  $r < 1$ , we should accept with probability  $r$ . This is achieved by drawing a value ( $u$ ) from a uniform distribution between 0 and 1. Because it is uniform, the probability that  $u$  will be less than or equal to  $r$  is equal to  $r$ . Hence, we accept the proposal if  $u \leq r$  and reject otherwise. The whole process is best illustrated with an example.

Figure 4.11 shows the Metropolis–Hastings algorithm in action, sampling from an arbitrary density (indicated by contours) (MATLAB script: `mhexample.m`). The starting point,  $\mathbf{w}_1$  is shown in Figure 4.11(a). Our proposal density is Gaussian with  $\Sigma = \mathbf{I}$ . From the starting point, the first proposal made is  $\tilde{\mathbf{w}}_2$ , shown in Figure 4.11(b). The proposal causes an increase in posterior density and is therefore accepted:  $\mathbf{w}_2 = \tilde{\mathbf{w}}_2$ . This acceptance is indicated by the solid line in Figure 4.11(b). The next proposal  $\tilde{\mathbf{w}}_3$  causes a slight decrease in posterior density but is accepted nonetheless (remember that, if the proposal causes a decrease, there is still a probability of acceptance). This is shown by the new solid line in Figure 4.11(c). The next proposal  $\tilde{\mathbf{w}}_4$  causes a large decrease in the posterior density value. Such a proposal is highly unlikely to be accepted (the ratio is much less than 1) and in this instance it isn't. This is represented by the dashed line in Figure 4.11(c). Hence,  $\mathbf{w}_4 \neq \tilde{\mathbf{w}}_4$  and is instead set to  $\mathbf{w}_4 = \mathbf{w}_3$ . This process continues in Figures 4.11(d) and 4.11(e) by which time we have ten samples. Along the way, three proposals were rejected and in each of those instances the sample is set to be equal to the value of the previous (accepted) sample. Continuing this process, we can see the first 300 accepted samples in Figure 4.11(f). These samples look reasonably consistent with the density contours – samples seem to be more concentrated towards the centre of the density and very sparse around the edges.

The density we are sampling from in this example happens to be a Gaussian. So, we can go some way towards convincing ourselves that we are indeed sampling from the correct density by computing the mean and covariance of the samples and seeing if they correspond to the mean and covariance of the actual density. The actual mean and covariance are given by

$$\boldsymbol{\mu} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{S} = \begin{bmatrix} 3 & 0.4 \\ 0.4 & 3 \end{bmatrix}$$

After  $N_s = 10,000$  samples, we can compute the sample-based approximations to the mean and covariance ( $\boldsymbol{\mu}', \mathbf{S}'$ ) as follows:

$$\boldsymbol{\mu}' = \frac{1}{N_s} \sum_{s=1}^{N_s} \mathbf{w}_s, \quad \mathbf{S}' = \frac{1}{N_s} \sum_{s=1}^{N_s} (\mathbf{w}_s - \boldsymbol{\mu}')(\mathbf{w}_s - \boldsymbol{\mu}')^T.$$

These work out as

$$\boldsymbol{\mu}' = \begin{bmatrix} 0.9770 \\ 1.0928 \end{bmatrix}, \quad \mathbf{S}' = \begin{bmatrix} 3.0777 & 0.4405 \\ 0.4405 & 2.8983 \end{bmatrix}$$

which are both very similar to the true values.

Before we move on to applying MH to our binary response model, we need to discuss two related concepts – **burn-in** and **convergence**. As we can start our sampler from anywhere (there is no restriction on  $\mathbf{w}_1$ ), we don't necessarily know if we are starting the sampler in an area that we should be generating samples from (it might be an area of very low posterior density). Therefore, the first few samples may not be representative and should be discarded. This period between the starting point and convergence of the sampler is known as the burn-in period. Sadly, it is not possible to conclusively determine how long this period should be. In the example described above, it is no more than a couple of samples but in some applications it could easily be hundreds or thousands. To overcome this problem we need a method for determining convergence. This is not convergence to a particular *value*, but convergence to a particular *distribution*. In other words, are the samples we are seeing coming from the correct distribution?

A popular method is to start several samplers simultaneously from different starting points. When all of the samplers are generating samples with similar characteristics (mean, variance, etc.), it suggests that they have all converged to the same distribution – the one we are trying to sample from.

We will now return to our binary response model. Using the MH scheme described above, we generate 10,000 samples from  $p(\mathbf{w} | \mathbf{X}, \mathbf{t}, \sigma^2)$  (MATLAB script: `logmh.m`). Our proposal density is a Gaussian with  $\Sigma = \mathbf{I}$  where  $\mathbf{I}^2 = 0.5$ . In Figure 4.12(a) we show every tenth sample (plotting all 10,000 samples makes for a very crowded plot) along with the posterior contours. The samples and the contours look reasonably coherent. If we like, we can use the samples to create marginal posterior densities for the two individual parameters. Recall from Section 2.2.6 that to marginalise  $w_2$  from the posterior we would need to integrate (or sum, if the random variable is discrete) over all values  $w_2$  could take:

$$p(w_1 | \mathbf{X}, \mathbf{t}, \sigma^2) = \int p(w_1, w_2 | \mathbf{X}, \mathbf{t}, \sigma^2) dw_2$$

where  $p(w_1, w_2 | \mathbf{X}, \mathbf{t}, \sigma^2)$  is another way of writing  $p(\mathbf{w} | \mathbf{X}, \mathbf{t}, \sigma^2)$ . To get a sample-based approximation, we take each of our samples,  $\mathbf{w}_s$ , and ignore  $w_2$ . In other words, if we throw away the value of  $w_2$  from each sample, we are left with a set of samples from  $p(w_1 | \mathbf{X}, \mathbf{t}, \sigma^2)$ . In Figures 4.12(b)–4.12(d), we show three popular ways of visualising these samples. In the first, (Figure 4.12(b)) we have split the range of possible values into 20 sections and counted the number of samples that fall in each section. The black bars show the numbers for  $w_1$  and the grey bars for  $w_2$ . If we were to take the number of samples falling into a particular section and divide it by the total number of samples, the numbers obtained could be thought of as the posterior probabilities that the  $w_1$  (or  $w_2$ ) falls into each of these sections. In the second example (Figure 4.12(c)), we have just plotted all 10,000 samples for  $w_1$  (a similar plot for  $w_2$  looks almost identical). This plot gives us confidence that the sampler has converged very quickly. If it hadn't, we might see an overall increasing or decreasing trend. In Figure 4.12(d) we show two continuous density functions that have been fitted to the samples. This is, in itself, a machine learning task for which there are various possible solutions. If the samples looked like they had come from a Gaussian, we could fit Gaussian densities to the two sets of samples (see Exercise 2.8). In this example, we have used a more general technique known as kernel density estimation. This can be performed in MATLAB using the `ksdensity` function. We won't go into any more detail here – the important point is that there are many ways to visualise the samples and it is possible to turn them into (approximate) continuous density functions.

Finally we turn our attention back to the predictive probability,  $P(T_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \mathbf{X}, \mathbf{t}, \sigma^2)$ . When using the Laplace approximation, we approximated this quantity by drawing samples  $\mathbf{w}_1, \dots, \mathbf{w}_{N_s}$  from the approximate posterior and then computing

$$P(T_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \mathbf{X}, \mathbf{t}, \sigma^2) = \frac{1}{N_s} \sum_{s=1}^{N_s} \frac{1}{1 + \exp(-\mathbf{w}_s^T \mathbf{x}_{\text{new}})}.$$

We now have a set of samples from the true posterior,  $p(\mathbf{w} | \mathbf{X}, \mathbf{t}, \sigma^2)$  and we can use them in exactly the same way. Figure 4.12(e) shows the predictive probability contours computed using these true posterior samples. Remember that these contours give the probability of classifying an object at any particular location as a square. The shape of the contours looks rather like the shape in Figure 4.7(b), which is not very surprising, as we saw in Figure 4.6(b) that the Laplace approximation didn't look too different from the true posterior. The only noticeable difference is that the contours in Figure 4.12(e) are slightly less tightly curved around the areas in which the data lie. This suggests that the probability reduces rather more slowly as we move away from the squares. The MH sampler is sampling from the true posterior and so the contours in Figure 4.12(e) should be considered closer to the truth than those for the Laplace approximation in Figure 4.6(a). This comparison is really just giving us an indication of how good the Laplace approximation is at making predictions. Figure 4.12(f) shows the decision boundaries corresponding to 20 of the MH samples picked at random (c.f. Figure 4.7(a)) (see Exercises 4.6, 4.7 and 4.8).

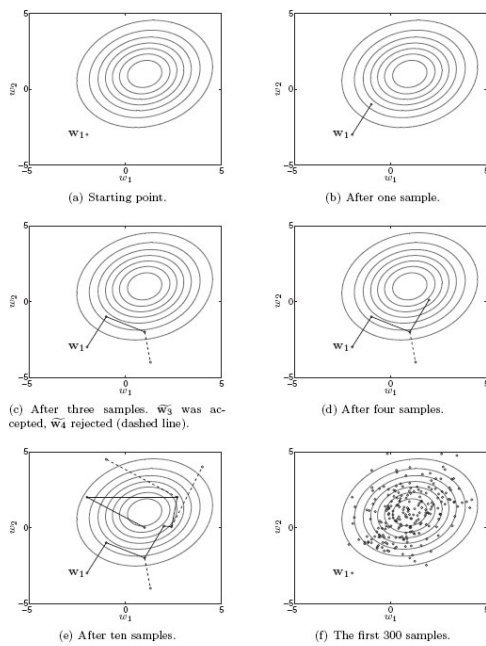


FIGURE 4.11 Example of the Metropolis-Hastings algorithm in operation.

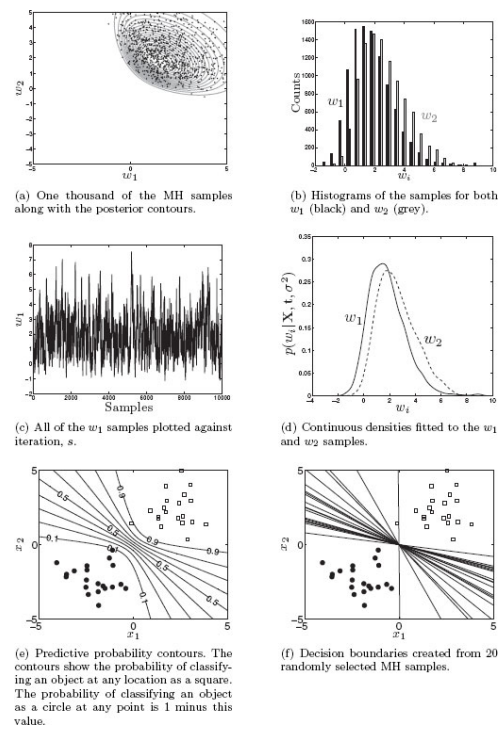


FIGURE 4.12 Results of applying the MH sampling algorithm to the binary response model.

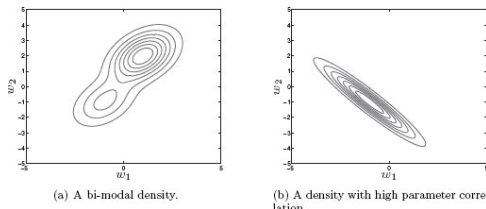


FIGURE 4.13 Two densities that would be tricky to sample with MH.

#### 4.5.3 The art of sampling

The Metropolis-Hastings algorithm seems to work well for our binary response model. This will not always be the case – sampling methods (like MH) can be tricky to use. The difficulty lies in the (often unknown) shape of the density from which we are attempting to sample. Consider the density shown in Figure 4.13(a). This density has two modes, one at  $\mathbf{w} = [-1, -1]^T$  and one at  $\mathbf{w} = [2, 2]^T$ . MH likes to move towards the modes, as these are moves that increase the posterior density and are therefore always accepted. Imagine  $\mathbf{w}_s$  somewhere near the mode at  $\mathbf{w} = [2, 2]^T$ . To move from here to the mode at  $\mathbf{w} = [-1, -1]^T$  would require many downhill steps in a row. Although this is possible, it is incredibly unlikely. This is where the relationship between the theory and practice of algorithms like MH breaks down. In theory, we will move from the top of one mode to the top of the other at some point in the future (just because it is unlikely doesn't mean that it won't happen eventually). In practice, we might get very old while we wait. We will happily explore one of the modes without ever realising that the other exists.

A second problem is illustrated by the density in Figure 4.13(b). Here, we have a density with only one mode but the two variables  $w_1$  and  $w_2$  are very dependent or highly correlated. If the value of  $w_1$  is known, it is possible to narrow  $w_2$  down to quite a small range. Densities such as this make it very difficult to choose proposal distributions:  $p(\mathbf{w}_s | \mathbf{w}_{s-1})$ . Pick any position on the density in Figure 4.13(b) and imagine proposing a movement based on a Gaussian density with a diagonal covariance matrix (as we used in all of our examples) which would have circular contours when plotted. The density shown in Figure 4.13(b) is far from circular, and, because the shape of the proposal density is so different from the shape of the density we are attempting to sample from, many samples will be rejected: the vast majority of moves that we sample from our proposal will involve moving steeply down the probability gradient.

These are not the only problems. For example, how do we know when we have taken enough samples? How do we know how many samples we need to discard at the beginning? Fortunately, there are ways to overcome all of these problems: more sophisticated algorithms, ways of choosing proposal densities, quantities that we can evaluate that indicate convergence, etc. Further details are provided in the suggested reading.

## 4.6 CHAPTER SUMMARY

The motivation for this chapter was the desire to do things in the Bayesian way when we are not able to compute the distributions of interest analytically. We have shown examples of three general techniques. Firstly, finding the highest point of the posterior (the MAP estimate). This is a single value, and single values are not very Bayesian but it incorporates the prior and

could therefore be considered a step up from the maximum likelihood solution. The second approach was to approximate the posterior with another density. We chose the Laplace approximation which approximates the posterior with a Gaussian. In many applications, this density could be used to compute the required expectations (predictions) analytically. In our binary response application, the expectation was not analytically tractable, but sampling from a Gaussian is easy and so we approximated (again) with a sample-based approximation. The third approach involved using the Metropolis-Hastings algorithm to generate samples from the true posterior which could be used to compute expectations. This comes at additional computational cost but (in theory at least) we get predictions that reflect the true posterior.

## 4.7 EXERCISES

- 4.1 For a data set consisting of  $N$  observations  $\mathbf{x}_n$  (each of which is  $D$ -dimensional) and *real* valued targets  $t_n$ , a linear regression model is defined as

$$p(t_n | \mathbf{x}_n, \mathbf{w}) = \mathcal{N}(\mathbf{w}^T \mathbf{x}_n, 1).$$

Making the standard IID assumption and assuming a Gaussian prior over the  $D$ -dimensional parameters  $\mathbf{w}$ , show that the Laplace approximation is equal to the true posterior.

- 4.2 In Chapter 3 we computed the posterior density over  $r$ , the probability of a coin giving heads, using a beta prior and a binomial likelihood. Recalling that the beta prior with parameters  $\alpha$  and  $\beta$  is given by:

$$p(r | \alpha, \beta) = \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} r^{\alpha-1} (1-r)^{\beta-1}$$

and the binomial likelihood, assuming  $y$  heads in  $N$  throws, is given by:

$$p(y | r, N) = \binom{N}{y} r^y (1-r)^{N-y},$$

compute the Laplace approximation to the posterior. (Note: You should be able to obtain a closed-form solution for the MAP value,  $\hat{r}$ , by setting the log posterior to zero, differentiating and equating to zero.)

- 4.3 Plot the true beta posterior and the Laplace approximation computed in Exercise 4.2 for various values of  $\alpha$ ,  $\beta$ ,  $y$  and  $N$ .

- 4.4 Given the expression for the area of a circle,  $A = \pi r^2$ , and using only uniformly distributed random variates, devise a sampling approach for computing  $\pi$ .

- 4.5 Rearrange the logistic function

$$P(T_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \hat{\mathbf{w}}) = \frac{1}{1 + \exp(-\hat{\mathbf{w}}^T \mathbf{x}_{\text{new}})}$$

to show that  $P(T_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \hat{\mathbf{w}}) = 0.5$  implies  $\hat{\mathbf{w}}^T \mathbf{x}_{\text{new}} = 0$ .

- 4.6 Assume that we observe  $N$  vectors of attributes,  $\mathbf{x}_1, \dots, \mathbf{x}_N$  and associated integer counts  $t_1, \dots, t_N$ . A Poisson likelihood would be suitable:

$$p(t_n | \mathbf{x}_n, \mathbf{w}) = \frac{(\mathbf{w}_0 \mathbf{x}_n)^{t_n} \exp(-\mathbf{w}_0 \mathbf{x}_n)}{t_n!}.$$

Assuming a Gaussian prior on  $\mathbf{w}$ , derive the gradient and Hessian needed to use a Newton-Raphson routine to find the MAP solution for the parameters  $\mathbf{w}$ .

4.7 Derive the Laplace approximation for the model in [Exercise 4.6](#).

4.8 Implement a Metropolis-Hastings sampling scheme for the model of [Exercise 4.6](#) and compare the posterior with the Laplace approximation derived in [Exercise 4.7](#).

#### 4.8 FURTHER READING

- [1] Christophe Andrieu, Nando de Freitas, Arnaud Doucet, and Michael Jordan. An introduction to MCMC for machine learning. *Machine Learning*, 50:5–43, 2003.

A tutorial introduction to MCMC techniques from the specific point of view of Machine Learning.

- [2] Siddhartha Chib. Understanding the Metropolis-Hasting algorithm. *The American Statistician*, 49(4):327–335, 1995.

An excellent tutorial on the Metropolis-Hasting algorithm. A good starting point to look deeper into this family of algorithms.

- [3] Arnaud Doucet, Nando de Freitas, and Neil Godron, editors. *Sequential Monte Carlo Methods in Practice*. Springer, 2010.

We have not covered sequential Monte Carlo techniques in this book, but they are becoming increasingly popular for performing Bayesian inference in complex models, particularly models with a temporal component, like target tracking.

- [4] Andrew Gelman, John B. Carlin, Hal S. Stern, and Donald B. Rubin. *Bayesian Data Analysis*. Chapman and Hall/CRC, second edition, 2004.

This is an excellent resource for practical Bayesian inference. In particular, it provides a solid introduction to other sampling techniques as well as procedures for determining if Metropolis-Hastings and other sampling algorithms have converged.

- [5] W. R. Gilks, S. Richardson, and D. Spiegelhalter, editors. *Markov Chain Monte-Carlo in Practice*. Chapman and Hall/CRC, 2005.

An edited volume providing several interesting practical sampling examples.

- [6] Mark Girolami and Ben Calderhead. Riemann manifold Langevin and Hamiltonian Monte Carlo methods. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 73(2):123–214, 2011.

A recent paper by the authors describing a sophisticated Metropolis algorithm for sampling from distributions with complex forms.

- [7] Jun Liu. *Monte Carlo Strategies in Scientific Computing*. Springer, 2008.

- [8] Carl Rasmussen and Christopher Williams. *Gaussian Processes for Machine Learning*. The MIT Press, 2006.

In this chapter we briefly mentioned the non-parametric Gaussian process as an alternative to the parametric model.

This book provides a comprehensive introduction to the use of Gaussian processes for both classification and regression.

- [9] Simon Rogers, Richard Scheltema, Mark Girolami, and Rainer Breitling. Probabilistic assignment of formulas to mass peaks in metabolomics experiments. *Bioinformatics*, 25(4):512–518, 2009.

A paper by the authors that described an alternative Bayesian sampling method (Gibbs sampling) being applied to the problem of detecting metabolites in mass spectrometry experiments.

- [10] Michael Tipping. Sparse Bayesian learning and the relevance vector machine. *Journal of Machine Learning Research*, 1: 211–244, 2001.

An example of both regression and classification based on a linear model. The Laplace approximation is used in the classification example.

- [11] Christopher Williams and David Barber. Bayesian classification with Gaussian Processes. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 20(12):1342–1351, 1998.

One of the first papers to use Gaussian processes for classification. A good introduction into some real approximate Bayesian inference.

<sup>1</sup>Technically, it is actually a saddle-point approximation but has come to be known as the Laplace approximation within machine learning. In computational statistics, the Laplace approximation is a name given to something else entirely.

<sup>2</sup>Named after Nicholas Metropolis and W. Keith Hastings – a physicist and statistician, respectively, who developed the technique to tackle problems in an area of physics known as statistical mechanics.